

Contents

Tech Stack	2
Test Plan:	2
Error Contract	3
File structure	4
This app will be used mainly by HR to do the following:	10
How different modules depend on each other?	10
CRUD on employee	11
Salary components for employees →	11
Employees' Contracts →	11
Generate a payslip for an employee (calculations logic is done in the db code)	12
Manage employee loans	12
Manage employee leaves	13
Track employee attendance	13
Produce a report for all of the above	14
Business Logic in Database:	15
Bean Validation:	16
Security (Authentication & Authorization):	16

Tech Stack

Programming language → Java 25

Framework → Spring Boot

Project Manager → Maven

Dependencies → Spring Data JPA

MySQL Driver

Spring Security (JWT)

Library → MapStruct (to map entity to DTOs)

REST API specification → OpenAPI

Endpoints testing → Swagger

Database → MySQL

Test Plan:

Not yet defined; will address it as I gain more experience with Spring Boot testing

Error Contract

Every error response returns the same JSON shape, handled by one global `@RestControllerAdvice`:

```
{
  "timestamp": "2026-08-24T14:32:00",
  "status": 404,
  "errorCode": "EMPLOYEE_NOT_FOUND",
  "message": "Employee with id 42 not found",
  "path": "/api/employees/42",
  "fieldErrors": []
}
```

`fieldErrors` is only populated for validation failures (400), listing which field failed and why.

Status codes used:

- 400 — validation failure (bad request body)
- 401 — missing/invalid JWT
- 403 — valid JWT, wrong role
- 404 — resource not found
- 409 — conflict (e.g. deleting an employee with payroll history, approving an already-approved leave)
- 500 — unexpected server error

File structure

/mapper

One mapper class for each module

/config

SecurityConfig.java (Spring Security setup — add JWT filter chain here)
OpenApiConfig.java (Swagger/OpenAPI bean configuration)
JacksonConfig.java (date format, null handling, global ObjectMapper settings)

/exception

... handle exception such as EmployeeNotFound etc.

/organization (I believe that organization details should not be changed , so this is read-only)

/entity

/Branch

/Department

/JobTitle

/dto

/BranchRequest / BranchResponse

/DepartmentRequest / DepartmentResponse

/JobTitleRequest / JobTitleResponse

/repository

one for each entity

/service

one for each entity

/controller

one for each entity

/employees

/entity

/Employee

/EmployeeSalaryComponent

/EmployeeContract

/DTO

/EmployeeRequest

/EmployeeResponse

/EmployeeSalaryComponentRequest

/EmployeeSalaryComponentResponse

/EmployeeContractRequest

/EmployeeContractResponse

/Repository

... one for each entity (extends JPA Repository)

/service

... one for each entity

/controller

EmployeeController implements employee CRUD and /contracts. Sub-resources under /api/employees/{id}/... that belong to other modules — loans, leaves — are implemented by that module's own controller

/payroll (will add here everything needed to generate and calculate payslip)

/entity

/PayrollPayment (shows when and amount paid to employee – salary) (each employee each month)

/PayrollPeriod (it is the month – period – when we will pay the salary)(each month)

/PayrollSetting (fixed data doesn't change) (used in calculating other stuff)

/PayslipLine (parts of salary – additions and deductions)

/SalaryComponent (fixed components of salary by their names) (doesn't change)

/TaxBracket (use it in calculating taxes for each employee)

/Payslip (use all above data to generate it)

/dto

... request and response for:

Payrollpayment, payrollperiod, payslipline,payslip (CRUD)

/repository

One for each entity (done automatically by JpaRepository)

/service

One for each entity in DTO

/controller

One for each dto

/loans

/entity

/Loan (employee takes money from company)

/LoanInstallment (employee pays back that money in installments)

/dto

Request and response for each entity

/repository

One for each entity (CRUD)

/service

One for each entity

/controller

GET , POST, PATCH

/leaves (raseed el agazat)

/entity

/LeaveType (fixed not changed)

/LeaveRequest (employee requests for a leave)

/LeaveBalance (shows how many leaves – days – remaining for employee to take off annually based on leave requests made)

/dto

Response and request for LeaveRequest and LeaveBalance

/repository

For LeaveRequest and for LeaveBalance

/service

For LeaveRequest and for LeaveBalance

/controller

For each employee → GET, POST LeaveRequest; GET LeaveBalance

/attendance

/entity

AttendanceSummary

/dto

Request and response for AttendanceSummary

/repository

For AttendanceSummary

/service

For AttendanceSummary

/controller

AttendanceSummary

/report (only to read report so no need for an entity)

/dto

One for each report

/repository

One repo

/controller

Containing the GET endpoints

This app will be used mainly by HR to do the following:

- CRUD an employee
- Generate a payslip for an employee
- Manage employee loans
- Manage employee leaves
- Track employee attendance
- Produce a report for all of the above

How different modules depend on each other?

- Employee depends on Organization (employee works in a branch in a department and has a job title)
- Loan, Leaves, Attendance depend on Employee (employee ID)
- Payroll depends on Employee, Loan, Leaves, Attendance

CRUD on employee

Create → POST/api/employees (should pass the employee contract data as well)

Read → GET/api/employees GET/api/employees/{id} GET/api/employees{id}/contracts

Update → PUT/api/employees/{id}

DELETE → POST/api/employees/{id}/terminate (based on sql code, I cant delete an employee if he has a payslip record, so this endpoint will set the status to TERMINATED and the db will do the rest of the work)

Salary components for employees →

GET /api/employees/{empId}/salary-components (read salary component for an employee)

POST /api/employees/{empId}/salary-components (create salary components for an employee)

PATCH /api/salary-components/{id} (update salary component)

DELETE /api/salary-components/{id} (delete salary component)

Employees' Contracts →

GET /api/employees/{empId}/contracts (read contract of an employee)

POST /api/employees/{empId}/contracts (renew contract of an employee)

PATCH /api/contracts/contractID (to end a contract)

(when a new contract is created for an employee, the old active contract will be automatically closed (status=EXPIRED) in the same transaction, so an employee should never have 2 active contracts at the same time.)

Generate a payslip for an employee (calculations logic is done in the db code)

HR generates a payslip for each employee at end of month → POST /api/payroll-periods/{periodCode}/run (this should excute db built in function sp_run_payroll)

HR marks payslips as paid → POST /api/payroll-periods/{periodCode}/pay

HR reads all payroll periods → GET /api/payroll-periods

HR reads one payroll period → GET /api/payroll-periods/{periodCode}

HR reads one payslip → GET /api/payslips/{payslipId}

HR reads all payslips for a specific month → GET /api/payslips?periodCode=yyyy-mm

HR reads one payslip broken down (lines) → GET /api/payslips/{payslipID}/lines

HR reads tax brackets and payroll settings → GET /api/tax-brackets?fiscalYear=2026
GET /api/payroll-settings?fiscalYear=2026

Manage employee loans

HR reads employee loans → GET /api/loans GET /api/loans/{loanId}

HR creates a new employee loan → POST /api/loans

HR reads loans for one employee → GET /api/employees/{empID}/loans

HR closes a loan early (manual payoff) → PATCH /api/loans/{loanId}

Manage employee leaves

HR reads leaves requests → GET /api/leaves GET /api/leaves/{requestId}

HR reads leaves requests for one employee → GET /api/employees/{empld}/leaves

HR creates a new leave request for an employee → POST /api/employees/{empld}/leaves

HR reads the leave balance for an employee → GET /api/employees/{empld}/leave-balances

HR reads a single leave balance → GET /api/leaves/leave-balances/{balanceId}

HR approve/rejects a leave request → POST /api/leaves/{requestId}/approve
POST /api/leaves/{requestId}/reject
{"rejectReason": "some reason"}

HR cancels leave request (leave balance stays the same) → POST /api/leaves/{requestId}/cancel

(I think leave types should not be altered)

Track employee attendance

HR reads the whole attendance sheet → GET /api/attendance

HR reads the attendance of one employee → GET /api/employees/{empld}/attendance

(I assumed that attendance data is passed automatically into the db by a biometric device or something)

Produce a report for all of the above

GET /api/reports/employee-directory → report for all employees info

GET /api/reports/headcount-by-department → no of employees and avg salary per department

GET /api/reports/payroll-register?periodCode=2026-07 → payslip for each employee in one month (period)

GET /api/reports/payroll-cost-by-department?periodCode=2026-07 → total company cost per department

GET /api/reports/payroll-trend

GET /api/reports/tax-insurance-liability?periodCode=2026-07

GET /api/reports/leave-balances?fiscalYear=2026&leaveType=ANN
→ Remaining leave days per employee for a given year/type

GET /api/reports/leave-requests?status=PENDING → view leave request that aren't approved yet

GET /api/reports/absence-watchlist

GET /api/reports/active-loans

GET /api/reports/contracts-expiring?months=12

GET /api/reports/overtime-top10?periodCode=2026-07 → top 10 employees by overtime and pay in a specific period

GET /api/reports/bank-transfer?periodCode=2026-07 → bank transfer file for each month

Business Logic in Database:

I decided to keep the payroll calculations (sp_run_payroll, sp_pay_period, fn_income_tax, fn_insurance_wage) as stored functions in the database, instead of rewriting them in Java.

Why: because they were already given with the dataset and they works correctly on the previous months. Also, the whole payslip (earnings, insurance, tax, deductions, loan installment) gets calculated in one database transaction, so I don't risk half-finished results.

Risks I'm aware of: the actual calculation rules aren't visible in the Java code; you'd have to open the SQL file to see them. Also, I can't unit test this logic, only test it against a real database. And the app becomes tied to MySQL.

How I'm handling it: the functions stay in erp_hr_payroll.sql and get committed to the Git repo with the rest of the code, not typed by hand into the database. Any change to the calculation logic means editing this file first, then re-applying it. The Java side only calls the function; it never repeats the logic:

```
public void runPayroll(String periodCode) {  
    entityManager.createNativeQuery("CALL sp_run_payroll(:code)")  
        .setParameter("code", periodCode)  
        .executeUpdate();  
}
```

And /run will return a summary (how many payslips were generated, updated period status, any employees skipped and why) instead of just an empty 200 OK.

Bean Validation:

Will use it with all entities to ensure data is valid

Security (Authentication & Authorization):

This app is to be used by the HR team only. I will assume that an HR team member can either be an HR_ADMIN or HR_USER

HR_ADMIN → view everything, run payrolls, approve loans and leaves

HR_USER → can view only (GET endpoints only)

add Spring Security dependency in pom.xml file

add JWT dependency in pom.xml file

have an entity for HR that contains each HR member's: id, role, username, password

HR logs in via POST /api/auth/login and passes in the details above; this should return the JWT token

Use BCryptPasswordEncoder to encrypt passwords